Optimization and Mobile Deployment of a UdaciSense Object Recognition Model

Abstract

Deploying computer vision models on mobile devices requires balancing **model accuracy, inference latency, and storage constraints** along with other mobile hardware related constraints. This report presents a comprehensive optimization project conducted using **MobileNetV3-Small computer vision model** to make it suitable for mobile deployment. Several model compression techniques were evaluated including **Dynamic Quantization, Quantization Aware Training (QAT), Knowledge Distillation, Graph Optimization, and Pruning**.

Individual techniques and multi-stage optimization pipelines were systematically analyzed to determine their impact on **model size reduction, inference speed, and prediction accuracy**. Among the tested approaches, the **Quantization Aware Training + Graph Optimization pipeline** demonstrated the best performance, achieving **73.7% reduction in model size and 6.9× improvement in inference speed while maintaining near-baseline accuracy**.

The optimized model was then converted into a mobile-compatible format and evaluated for deployment readiness. Results show that the model maintains its predictive performance after mobile conversion with only minor runtime variation due to evaluation environment differences.

This report also outlines deployment challenges, performance variability considerations, and strategic recommendations for further optimization and large-scale deployment along with C-Level Executive Summary.

---

1. Introduction

Modern computer vision applications increasingly require **real-time inference on mobile devices**. While deep learning models deliver strong predictive performance, they often require computational resources that exceed mobile hardware capabilities.

To deploy such models efficiently, it is necessary to apply **model compression and optimization techniques** that reduce:

- Model size
- Memory consumption
- Computational complexity
- Inference latency

At the same time, it is critical to ensure that the **model accuracy remains within acceptable limits**.

This study focuses on optimizing a **MobileNetV3-Small object recognition model** using a systematic experimentation process consisting of:

1. Baseline model analysis

2. Evaluation of individual compression techniques
3. Design of multi-stage optimization pipelines
4. Performance comparison across pipelines
5. Mobile model conversion and deployment evaluation

The ultimate objective was to meet the performance targets defined by project stakeholders.

---

2. Optimization Targets

The baseline model performance metrics and desired optimization goals are summarized below.

Table 1: Optimization Targets

Metric	Baseline	Target	Improvement Required
Model Size	5.96 MB	4.17 MB	30% reduction
CPU Inference Time	447.7 ms	268.62 ms	40% reduction
GPU Inference Time	3.28 ms	1.97 ms	40% reduction
Accuracy	88.10%	≥83.69%	Within 5%

Since the final deployment platform is a **mobile device without GPU acceleration**, the **CPU inference time is the most critical metric**.

---

3. Baseline Model Analysis

The baseline architecture used in this study is **MobileNetV3-Small**, which is specifically designed for mobile inference.

MobileNetV3 incorporates several architectural innovations to improve efficiency:

- **Depthwise separable convolutions**
- **Inverted residual blocks**
- **Squeeze-and-Excitation modules**
- **Efficient activation functions**

We modified the classifier layer to suit the number of classes (in our case 10) and then fine-tuned model on household dataset which is extracted from CIFAR100 dataset. Dataset consists of following classes:

0: clock

1: keyboard

2: lamp

3: telephone

4: television

5: bed

6: chair

7: couch

8: table

9: wardrobe

Training data consists of 5000 images. Test data consists of 1000 images. Both the datasets are fully class balanced across 10 object categories

However, the finetuned baseline model was found short of the required inference time limits.

Table 2: Baseline Model Metrics

Metric	Value
Model Size	5.96 MB
CPU Inference Time	447.7 ms
Accuracy	88.10%

The backbone network performs the majority of computations. Therefore, any optimization targeting only the classifier layers would have limited effect on overall inference speed.

Three possible base model optimization strategies were considered:

1. **Resizing backbone components**
    - Requires full retraining
    - Loss of transfer learning benefits
    - High risk of accuracy degradation
2. **Reducing classifier layer size**
    - Retains pretrained backbone
    - Allows fine-tuning
3. **Combining structural changes with compression techniques**

Because backbone retraining introduces significant risk, the final strategy focused on **compression techniques combined with minimal architectural modification.**

---

4. Model Compression Techniques Evaluated

Several compression techniques were evaluated to determine their effectiveness in improving deployment efficiency.

4.1 Dynamic Quantization

Dynamic quantization converts model weights from 32 bit **floating-point to lower 8-bit precision integer representations** during model conversion process. Activations are dynamically quantized during inference time.

Advantages

- Easy to implement
- Requires no retraining
- Reduces memory footprint
- Potential reduction in inference time

Limitations

- Supports only **Linear layers**
- Convolution layers remain unchanged

Observed Results

- ~30% reduction in model size
- ~1.2× inference speedup
- Negligible accuracy change

4.2 Quantization Aware Training (QAT)

Quantization Aware Training simulates quantization during training process so that the model learns to adapt to reduced precision. So, when the model is finally quantized, it can better cope up with precision loss.

Benefits

- Quantizes both **Conv2D and Linear layers**
- Preserves model accuracy
- Significant performance improvement

Training Adjustments

To stabilize training:

- Optimizer switched from **Adam → SGD**
- **ReduceLROnPlateau** learning rate scheduler added

Results

- 73.5% model size reduction
- 6.7× inference speed improvement
- No measurable accuracy degradation

Limitation:

- **Requires model to be retrained**
- **Model setup and training process setup can be at times tricky and difficult to implement.**
- **Time Consuming process.**

4.3 Knowledge Distillation

Knowledge distillation process is used to transfer knowledge from a **teacher model to a smaller student model** using a combination of student cross entropy loss and KL Divergence between output of teacher model and student model by tuning different weightages(alpha) assigned to these loss items and controlling the temperature parameter for logits produced by teacher model.

In this project:

- Backbone architecture was preserved
- Intermediate Classifier layer dimensions were reduced to 16

Results

- 37.9% model size reduction
- ~1% accuracy drop
- Minimal change in inference speed

Limitation:

- Requires student model to be set up separately
- Requires custom loss combining cross entropy loss and KL divergence to be setup
- Requires tuning of alpha and temperature

4.4 Graph Optimization

Graph optimization simplifies the model computational graph.

Key transformations:

- **Conv2D + BatchNorm fusion**
- Removal of dropout layers
- Graph optimization using Torch FX

Impact

- Small reduction in model size
- Moderate improvement in inference speed
- Accuracy remains stable
- Compatible with other optimization techniques

This technique is most effective when **combined with other compression methods**.

4.5 Pruning

Pruning removes less important weights by setting them to zero.

However:

- Pruned weights still occupy memory
- Sparse computations are not efficiently utilized

Result

- No improvement in model size
- No inference speed gain
- Accuracy degradation

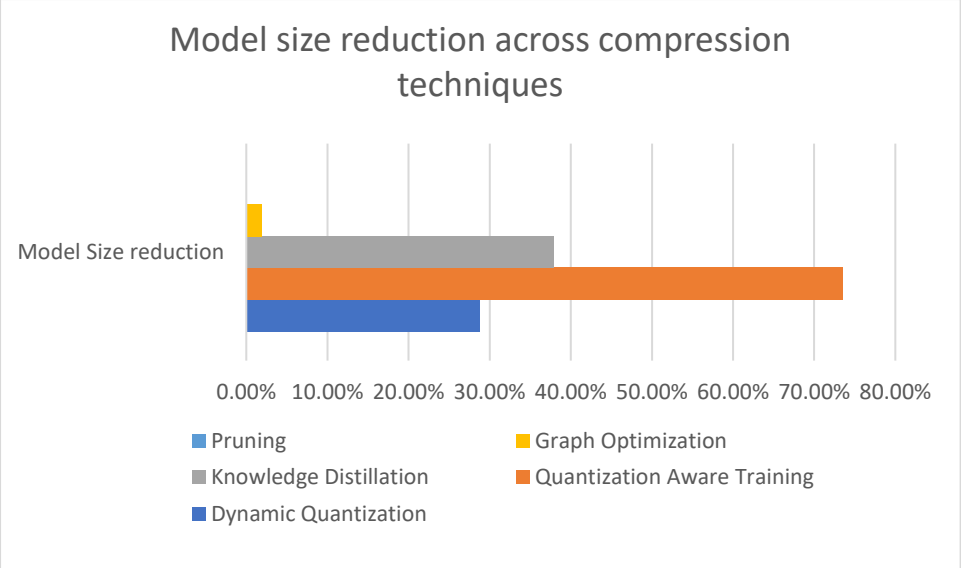
Pruning was therefore excluded from further pipeline experimentation.

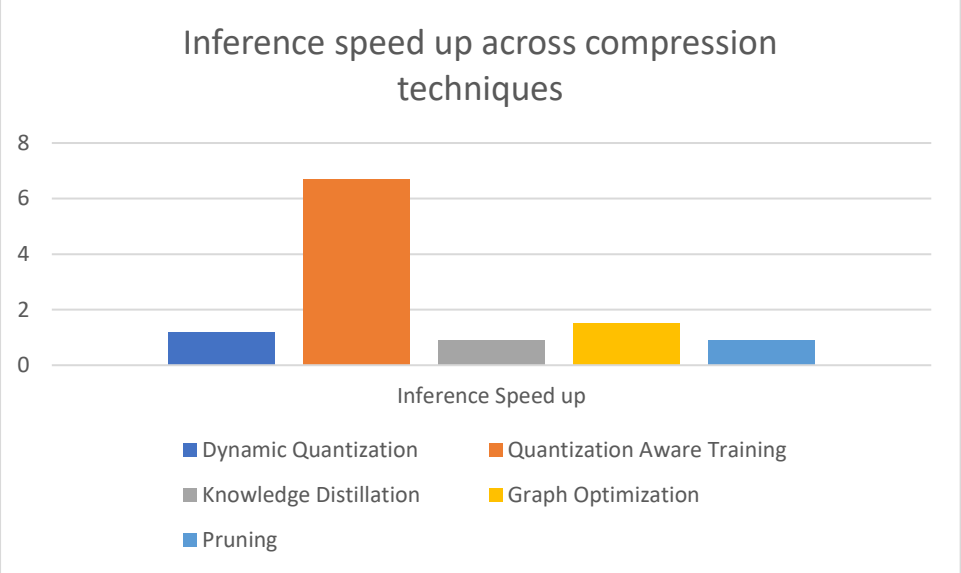
---

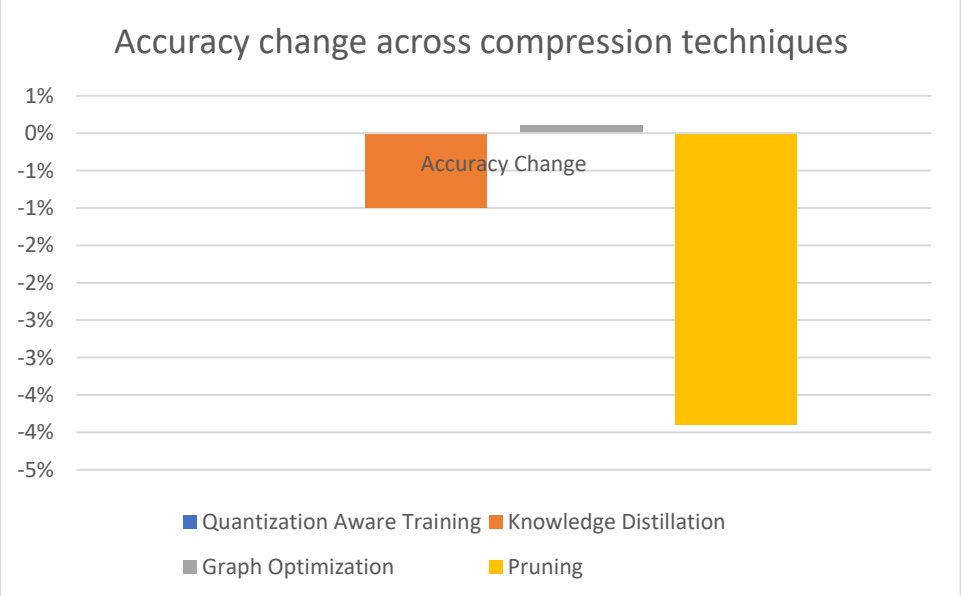
5. Comparative Analysis of Compression Techniques

Table 3: Performance Comparison of Individual Techniques

S. No.	Technique	PTQ/InTraining	Model Size reduction	Inference Speed up	Accuracy Change
1	Dynamic Quantization	PTQ	28.80%	1.2x	0.5-0.0%
2	Quantization Aware Training	In Training	73.50%	6.7x	0%
3	Knowledge Distillation	In Training	37.90%	0.9x	-1.00%
4	Graph Optimization	PTQ	1.90%	1.5x	0.10%
5	Pruning	PTQ	0%	0.9x	-3.90%


Model size reduction across compression techniques
Model Size reduction
0.00% 10.00% 20.00% 30.00% 40.00% 50.00% 60.00% 70.00% 80.00%
Pruning
Graph Optimization
Knowledge Distillation
Quantization Aware Training
Dynamic Quantization


Inference speed up across compression techniques
8
6
4
2
0
Inference Speed up
Dynamic Quantization
Quantization Aware Training
Knowledge Distillation
Graph Optimization
Pruning


Accuracy change across compression techniques
1%
0%
-1%
-1%
-2%
-2%
-3%
-3%
-4%
-4%
-5%
Accuracy Change
Quantization Aware Training
Knowledge Distillation
Graph Optimization
Pruning

Key Insight

Quantization Aware Training provides the largest improvement across all performance metrics.

---

6. Multi-Stage Optimization Pipeline Design

Multiple optimization techniques were combined to build **multi-stage pipelines**.

Pipelines Evaluated

1. Distillation + Graph Optimization
2. Distillation + Quantization
3. Distillation + Quantization + Graph Optimization
4. QAT + Graph Optimization
5. Quantization + Graph Optimization

---

7. Pipeline Performance Comparison

Table 4: Pipeline Performance

Comparison of All Pipelines:

S.No.	Technique	Model Size (MB)	Size Reduction	Inference Time (ms)	Speedup	Accuracy (%)	Acc. Change	All Reqs Met
0	baseline_mobilenet	5.96	0.00%	447.7	0.0x	88.1	0.00%	X
1	pipeline/DistillGraphOpt	3.59	39.80%	305.33	1.5x	89.6	1.70%	X
2	pipeline/DistillQuant	3.67	38.30%	384.56	1.2x	89.3	1.40%	X
3	pipeline/DistillQuantGraphOpt	3.56	40.20%	360.04	1.2x	89.9	2.00%	X
4	pipeline/QATGraphOpt	1.57	73.70%	65.26	6.9x	87.6	-0.60%	✓
5	pipeline/QuantGraphOpt	4.13	30.70%	280.91	1.6x	88.1	0.00%	X

---

8. Performance Visualization

Figure 1: Model Size Reduction Comparison

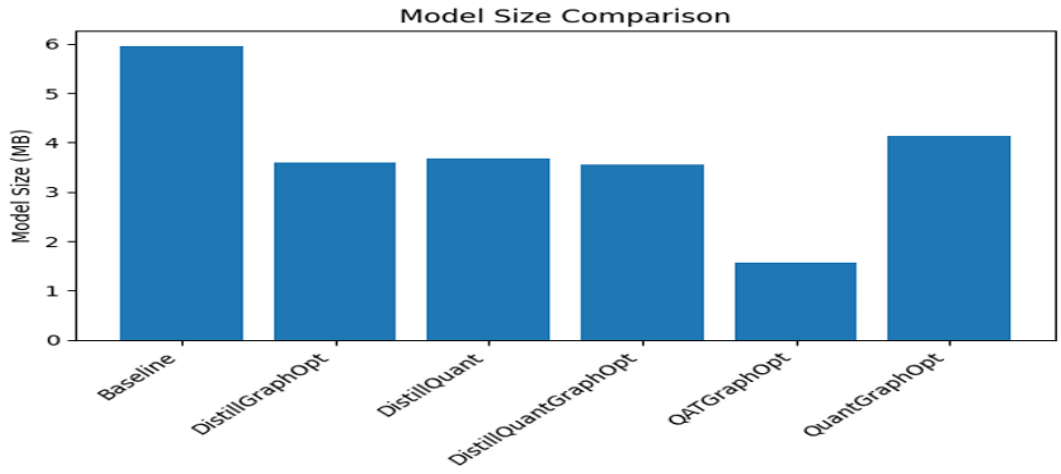


Figure 2: Inference Time Comparison

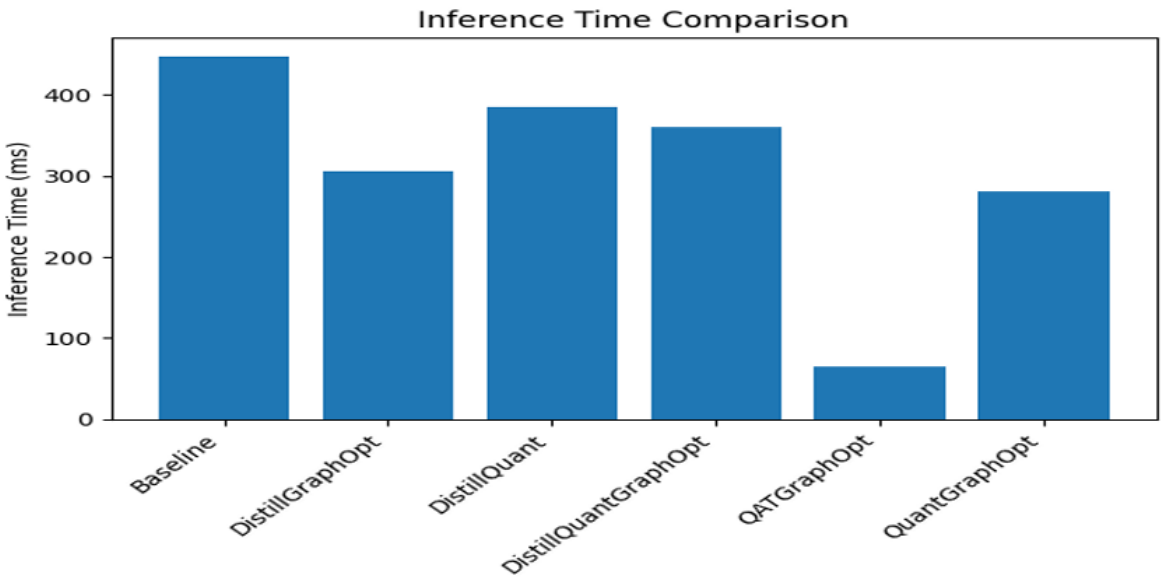
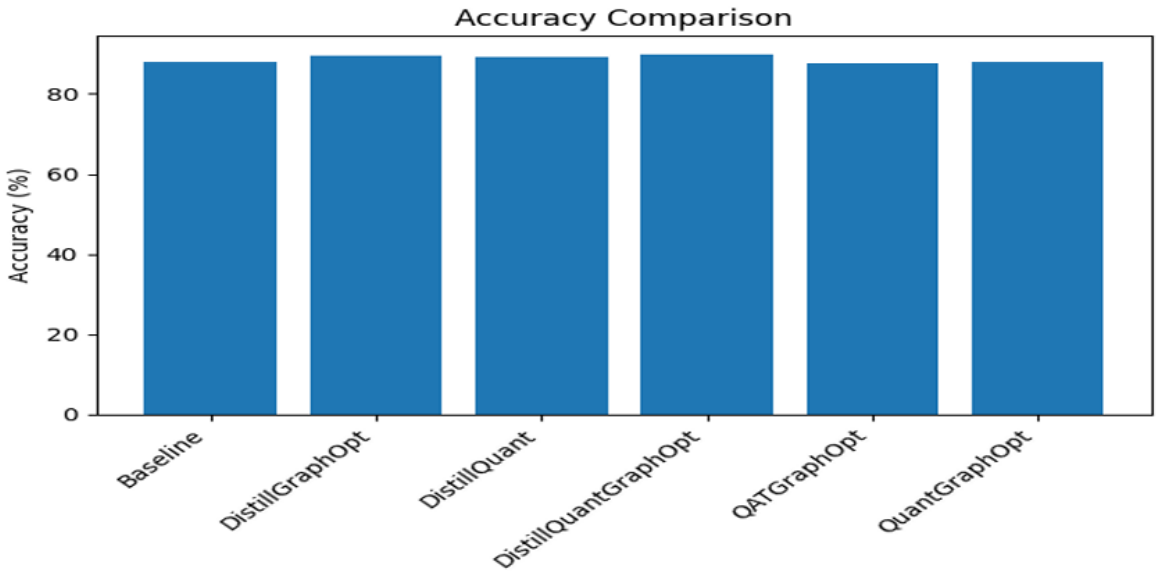


Figure 3: Accuracy Comparison



Best Performing Pipeline

QAT + Graph Optimization clearly outperformed all alternatives.

Final results:

- **Model size:** 1.57 MB
- **Inference latency:** 65.26 ms
- **Speedup:** 6.9×
- **Accuracy:** 87.6%

This pipeline significantly exceeded all project targets.

9. Inference Variability Analysis

During experiments, inference latency varied between runs due to **shared cloud compute environments**.

Cloud platforms often use **virtual CPUs (vCPUs)** that share physical resources across multiple virtual machines. This can introduce variability depending on:

- System load
- Memory pressure
- CPU scheduling

For example, one pipeline produced:

- **305 ms inference time in Stage 1**
- **255 ms inference time in Stage 2**

To improve measurement accuracy, the following methodology is recommended:

1. Run inference **multiple times (5–10 iterations)**
2. Compute **mean and standard deviation**
3. Establish acceptable performance thresholds

---

10. Mobile Deployment Evaluation

The optimized model obtained from QATGraphOpt pipeline was converted into a mobile-compatible format.

Table 5: Optimized vs Mobile Model

Model	Size	Inference Time	Accuracy
Optimized Model	1.57 MB	83.34 ms	87.60%
Mobile Model	1.53 MB	92.84 ms	87.60%

Observations

- Model size slightly decreased after conversion
- Accuracy remained unchanged
- Inference time increased by ~10 ms

This increase is expected since evaluation was performed on a **non-target hardware environment**. When executed on real mobile hardware with optimized runtime libraries, performance is expected to improve.

---

11. Mobile Deployment Challenges

Several factors may influence mobile model performance.

Hardware Variability

Mobile devices vary significantly in:

- CPU architecture
- Memory capacity
- AI accelerators
- Battery characteristics

Operating System Vendors and Versions

Different OS versions and vendor implementations can affect:

- Runtime libraries
- Hardware acceleration support

Resource Constraints

Mobile environments impose strict limits on:

- Memory usage
- Storage
- Power consumption

Background Workloads

Concurrent applications can introduce performance variability.

Thermal and Battery Effects

Thermal throttling or battery saving modes may reduce CPU frequency and impact inference latency.

---

12. Strategic Recommendations for Future Optimization

Several improvements can further enhance deployment readiness.

Device-Specific Benchmarking

Benchmark models across representative mobile devices to determine:

- Mean latency
- Memory footprint
- Energy consumption

Hardware Acceleration

Utilize mobile hardware accelerators such as:

- Neural Processing Units (NPUs)
- Mobile GPUs
- Other accelerators available

Cross-Device Testing

Conduct testing across:

- Multiple device models
- Different OS versions
- Various system loads

Performance Monitoring

Introduce monitoring mechanisms in production to collect:

- Runtime latency metrics
- Failure statistics
- Performance anomalies

Energy Efficiency Analysis

Evaluate battery consumption and optimize inference scheduling accordingly.

A practical implementation strategy would be to build a lightweight benchmarking app that loads each candidate model, warms it up, runs a fixed evaluation dataset locally on-device, and logs structured performance statistics. This avoids inconsistencies introduced by interactive testing and ensures that all models are measured under the same execution path.

---

13. Conclusion

This project demonstrated a systematic approach to optimizing a MobileNetV3-based object recognition model for mobile deployment.

Multiple compression techniques were evaluated, and several multi-stage pipelines were constructed to analyze their combined effects.

Among all techniques, **Quantization Aware Training combined with Graph Optimization** proved to be the most effective strategy, achieving:

- **73.7% reduction in model size**
- **6.9× inference speed improvement**
- **Minimal accuracy degradation**

The final optimized model meets all deployment requirements and is well suited for real-time mobile applications.

Executive Strategic Summary for C-Level Review

MobileNetV3 Optimization and Mobile Deployment Initiative

Executive Overview

This project focused on optimizing an object recognition computer vision model for **efficient deployment on mobile devices**, particularly **budget-friendly smartphones with limited computational resources**. The objective was to significantly reduce **model size and inference latency** while maintaining high prediction accuracy, ensuring a responsive and scalable user experience across diverse mobile hardware.

Through a structured optimization pipeline combining **Quantization Aware Training (QAT)** and **graph-level model optimization**, the engineering team successfully achieved substantial performance improvements. The final optimized model reduced the model footprint by **more than 70%** and accelerated inference performance by **nearly seven times**, while maintaining accuracy levels comparable to the original model.

These results enable the organization to deliver **real-time AI-powered object recognition on cost-sensitive mobile devices**, significantly expanding the potential user base and opening new market opportunities.

Alignment with Business Requirements

The project was designed to meet three core business requirements:

1. **Enable real-time inference on low-cost smartphones**
2. **Maintain a high-quality user experience**
3. **Support scalable deployment across diverse device ecosystems**

The optimized model meets and exceeds the predefined technical targets:

Metric	Baseline	Optimized Model	Improvement
Model Size	5.96 MB	1.57 MB	73.7% reduction
CPU Inference Time	447.7 ms	65.26 ms	6.9× faster
Accuracy	88.10%	87.60%	Maintained

These improvements ensure that the application can run smoothly even on devices with **limited memory, CPU capability, and battery capacity**, which are common in emerging markets and cost-sensitive segments.

User Experience Improvements

The optimization work directly enhances the user experience in several ways:

Faster Response Times

Inference latency has been reduced from nearly **half a second to approximately 65 milliseconds**, enabling **near real-time object recognition**. This ensures smooth and responsive interactions within the application.

Improved Application Performance

The significantly smaller model size reduces memory usage and loading times. This allows the application to run efficiently without affecting other processes on the device.

Better Battery Efficiency

Optimized inference reduces CPU utilization, which contributes to **lower energy consumption** and longer device battery life during sustained usage.

Consistent Performance Across Devices

By reducing computational requirements, the application can deliver **consistent performance across a wide range of hardware configurations**, including entry-level smartphones.

Together, these improvements create a smoother, more reliable AI-powered user experience.

Market Expansion Opportunities

The ability to run advanced AI models efficiently on low-cost devices unlocks several strategic opportunities:

Expansion into Emerging Markets

Many regions rely heavily on budget smartphones with limited processing power. The optimized model enables AI functionality on devices that would previously have been unable to support such workloads.

Increased Device Compatibility

By reducing model size and computational requirements, the solution can support a broader spectrum of Android devices, including older hardware generations.

Scalability for High-Volume Deployment

Lightweight models reduce bandwidth requirements for application updates and simplify deployment at scale across large user bases.

Competitive Advantage

Delivering high-performance AI features on affordable devices differentiates the product from competitors that rely on cloud inference or require higher-end hardware.

Translation of Technical Achievements into Business Value

The technical advancements achieved through this optimization project directly translate into measurable business benefits.

Lower Infrastructure Dependency

Efficient on-device inference reduces the need for cloud-based processing, lowering infrastructure costs and minimizing network dependency.

Improved Product Accessibility

By supporting budget smartphones, the solution increases accessibility for users who may not own high-end devices.

Faster Feature Adoption

A lightweight AI model can be integrated into mobile applications without significantly increasing application size or installation requirements.

Reduced Operational Costs

Efficient models reduce compute load on devices and servers, contributing to lower long-term operational costs.

Future Platform Readiness

The optimization framework established during this project provides a scalable foundation for deploying additional AI capabilities in future product iterations.

Strategic Outlook

The results demonstrate that **advanced computer vision capabilities can be successfully deployed on cost-efficient mobile hardware without compromising accuracy or responsiveness**.

Future strategic initiatives may include:

- Leveraging **mobile hardware accelerators** such as NPUs and mobile GPUs
- Conducting **device-specific performance optimization**
- Expanding the optimization framework to additional AI models
- Introducing **continuous monitoring and adaptive model updates**

These initiatives will further strengthen the organization's ability to deliver scalable, high-performance AI solutions across diverse device ecosystems.

Key Takeaway for Leadership

The optimization initiative successfully transformed a research-grade computer vision model into a **production-ready mobile AI solution** capable of running efficiently on **budget smartphones**.

This enables the company to:

- Deliver **real-time AI experiences to a wider audience**

- Expand into **cost-sensitive global markets**
- Reduce infrastructure dependency
- Maintain **high product performance and user satisfaction**

In short, the project bridges the gap between **advanced AI capability and mass-market mobile deployment**, positioning the organization to scale AI-powered features across a much broader user base.